

CS3200 SEC001

**Student-Grade-Management-
System**

Programming Languages Project



Introduction/Background

- C++ is an Object Oriented language which includes principles such as classes, inheritance, encapsulation, abstraction, polymorphism, and bits of dynamic memory. The language was based on C and developed by Danish Computer Scientist - Bjarne Stroustrup at Bell Telephone Laboratories (now known as Nokia Bell Labs) in Murray Hill, New Jersey around 1983. The initial version of C++ was known as C with classes, and we currently have C++ 23 with features like static members with possibly C++ 26 underway?
- Typical applications of this language include Game engines (Unreal engine), Operating Systems (Windows OS), Cloud Computing platforms (AWS), etc.
- I choose C++ because Wayne State University's CSC are C++ heavy and emphasized in the beginning of the degree program. Courses include CS1100, CS1500 (Fundamental structures in computer science), CS2110 (Computer Science 1), etc.



Introduction/Background (2)



- C is **NOT** a object oriented programming language. C is a procedural general-purpose programming that was created by Dennis Ritches in 1972 within a lab of AT&T. The initial target application for this language was for a UNIX system, the pioneer of an OS where it's meant to be multiuser and multi tasking built strictly through the CLI unlike other OS's we see today. Due to it's low maintenance and powerful performance, C eventually became the meta when it comes to OS development.
- So are there any other applications that C specializes in? Yes! C worked with Databases (MySQL), Python interpreter, and Embedded Systems for its strong relationship with hardware.
- Wayne State University is not quite C heavy. However, courses like CS4420 Operating systems teaches students on how to use processes, threads, CPU scheduling and much more.

Language Features (1): Data types & variables

- In C, the data type string doesn't exist the way it does in C++. In C, strings are just an array of type char. This affects how we take user input for an undisclosed length of that string, and how we create an array of strings.

```
20 //How to create an array of string in c (must use a pointer)
21 char *arra[] = {"Iffat", "taffy"};
22 char *array[3] = {"Ifdf", "iwsh", "wedd"};
23 //printf("%s", arra[0]); //outputs Iffat
24
25 /*
26 char *name5;
27 printf("Enter your name: ");
28 scanf("%[^\n]%*c", name5);
29 printf("%s", name5);
30 */
31 arra[2] = "aq";
32 printf("%s", arra[2]);
33
34
35
36
```

```
8
9 #include <stdio.h>
10
11 int main()
12 {
13
14     //How to create a string in c
15     char name[] = "Iffat Rahman";
16     char *name2 = "Tadffy namhar";
17     char name3[5] = "Zacky";
18
19
20     //How to create an array of string in c (must use a pointer)
21     char *arra[] = {"Iffat", "taffy"};
22     char *array[3] = {"Ifdf", "iwsh", "wedd"};
23     //printf("%s", arra[0]); //outputs Iffat
24
25     char *name5;
26     printf("Enter your name: ");
27     scanf("%[^\n]%*c", name5);
28     printf("%s", name5);
29
30
31
32
```

input

Enter your name: Iffat Rahman
Iffat Rahman

...Program finished with exit code 0

Press ENTER to exit console

Language features (2): OOP – Structs, Classes, dynamic memory

```
typedef struct {
    void *Element; //pointer to our chosen datatype v
    size_t ElementSize; //size_t is an unsigned integ
    size_t Size; //Size will be used to determine th
    size_t Capacity;
    void (*Vector_Init)(struct Vector*, size_t);
    /*
    - the intialize function is kinda like a construc
    second parameter takes an argument for the variab
    the amount of bytes for the passed data type. You
    this is what is essentially the number of bytes w
    */
    void (*Vector_Resize)(struct Vector*, size_t);
    void (*Vector_Push_Back)(struct Vector*, void*);
    void* (*Vector_Get)(struct Vector*, size_t);
} Vector;
```

- In C vectors do not exist along with classes. A struct allows us to mimic classes, we implement a Vector struct. However, a struct only allows us to group severd related data members of different/similar data types along with pointers to the method (not the method itself). Classes allow us to group data members and methods allowing us to create our own data type.
- The Vector struct in c uses the keywords **malloc** (this is where we initialize or allocate memory on the heap to a pointer of some data type typically). However, if we want to re-initialize or give said pointer a different amount of contiguous memory we do so via **realloc**.

```
void Vector_Init(Vector *PTO, size_t ElementSize){
    PTO->Size = 0;
    PTO->Capacity = 4; //by default 4 is the raw # of bytes
    PTO->ElementSize = ElementSize;
    PTO->Element = malloc(PTO->Capacity * ElementSize);
}

//sizeof(respective data type)
void Vector_Resize(Vector *PTO, size_t Capacity){
    PTO->Capacity = Capacity;
    PTO->Element = realloc(PTO->Element, PTO->Capacity * PTO->ElementSize);
}

//&(respective data type)
void Vector_Push_Back(Vector* PTO, void *Element){
    if(PTO->Size == PTO->Capacity){
        Vector_Resize(PTO, PTO->Capacity * 2);
    }

    void *Slate = (char*)PTO + PTO->Size * PTO->ElementSize;
    memcpy(Slate, Element, PTO->ElementSize);

    PTO->Size++;
}

void* Vector_Get(Vector *PTO, size_t Index){
    return (char*)PTO->Element + Index * PTO->ElementSize;
}
```

```
//C FOR THE VECTOR STRUCT
Vector RegVecObject = {
    .Vector_Init = Vector_Init,
    .Vector_Resize = Vector_Resize,
    .Vector_Push_Back = Vector_Push_Back,
    .Vector_Get = Vector_Get,
};
```

Language features (3): Files

- Files in C:

- a) We use the standard library: `stdio.h`
- b) Similar to C++ we create an object type for the FILE struct, however we use pointers: `FILE *ptr;`
- c) We have to manually open the file before writing or reading from the file: `ptr = fopen("textfile.txt", "r or w");`

- Files in C++:

- a) We use the fstream library: `#include <fstream>`
- b) Similar to C we create an object type for the class of our choice to write/read: `ofstream/ifstream`
`FileObj("textfile.txt");`
- c) We don't have to manually open the file before writing or reading from the file.

Images of file usage between the two

C:

```
void LoadStudentRecord(Admin *PointerToObject){
    char Text[256];

    FILE *ptr = fopen("StudentData.txt", "r");

    while(fgets(Text, sizeof(Text), ptr) != NULL){
        printf("%s", Text);
    }

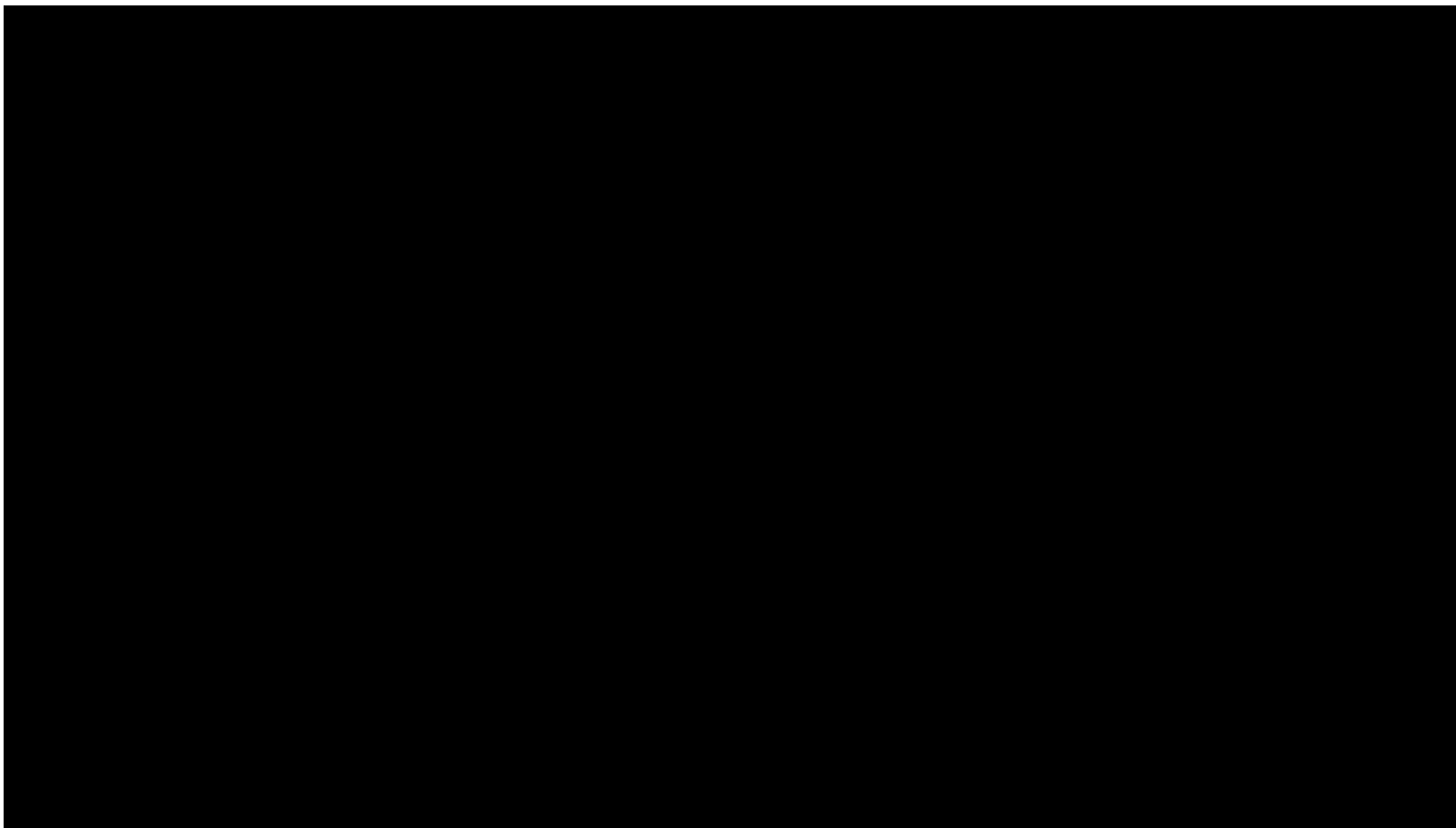
    fclose(ptr);
}
```

C++:

```
ifstream File("StudentData.txt");

while(getline(File, Text)){
    stringstream iss(Text);
    while(iss >> NumberFromLine){
        Placeholder = Placeholder + NumberFromLine;
        Counter++;
    }
}
```

Video of Live Demo



Sources:

- <https://www.geeksforgeeks.org/cpp/history-of-c/>
- <https://www.geeksforgeeks.org/cpp/cpp-23-standard/>
- <https://www.geeksforgeeks.org/linux-unix/introduction-to-unix-system/>
- <https://www.geeksforgeeks.org/c/history-and-application-of-c/>